9a) Surcharge d'opérateur

DinoVec2 operator+ (DinoVec2 a, DinoVec2 b)

DinoVec2 operator* (DinoVec2 a, float b)

operator* (float a, DinoVec2 b)

operator -

... etc ...

a + b ⟺ operator+ (a, b)

a * b ⟺ operator* (a, b)

→ Priorité non-changeable

a + b × c + d ⟺ operator+ (operator+ (a, operator* (b, c)), d)

Alternative

```
struct DinoVec2 {
    DinoVec2 operator+(DinoVec2 other);
    DinoVec2 operator*(float other);
}
```

vec2 + vec2 $\Longleftrightarrow$ vec2.operator+(vec2)

vec2 × float $\Longleftrightarrow$ vec2.operator*(float)

~~float × vec2 $\Longleftrightarrow$ float operator*(vec2)~~

Float n'a pas de méthodes, donc pas possible

Ce qui est écrit :

```
struct DinoVertex {
    DinoVec2 pos{};
    uint16_t u = 0;
    uint16_t v = 0;
    DinoColor color = DinoColor_WHITE;
};
```

→ Constructeur implicitement généré

Ce que le compilateur génère :

```
struct DinoVertex {
    DinoVec2 pos;
    uint16_t u;
    uint16_t v;
    DinoColor color;

    DinoVertex(); // CONSTRUCTEUR
};
```

→ Constructeur sans argument → Constructeur par défaut

```
DinoVertex::DinoVertex()
{
    pos = {};
    u = 0;
    v = 0;
    color = DinoColor_WHITE;
}
```

→ Avantage du constructeur : On ne peut pas oublier de l'appeler !

```
DinoVertex v; // Le compilateur appelle implicitement le constructeur
```

```
std::vector<DinoVertex> vs;
vs.resize(6);
```

↳ std::vector::resize() appelle le constructeur par défaut sur chaque nouvel élément

9c]

SYNTAXE DU CONSTRUCTEUR

```
class DinoPlayer {
public:
    DinoPlayer(———);
};
```

N'importe quels arguments
(si pas d'argument, "constructeur par défaut")

Le constructeur est une méthode avec le nom du type

Le constructeur n'a pas de type de retour

```
DinoPlayer::DinoPlayer(———){
———
}
```

g_Players.resize(4) → Ne compile plus car pas de constructeur par défaut

g_Players.emplace_back(_____)

↳ Arguments transférés au constructeur de l'élément

9d) En symétrie du constructeur, en C++, quand une variable est détruite (quand elle n'est plus accessible) le compilateur appelle automatiquement le destructeur

SYNTAXE DU DESTRUCTEUR

```
class DinoPlayer {
 public :
        ~DinoPlayer();
};
```

Aucun argument

Alt Gr + 2

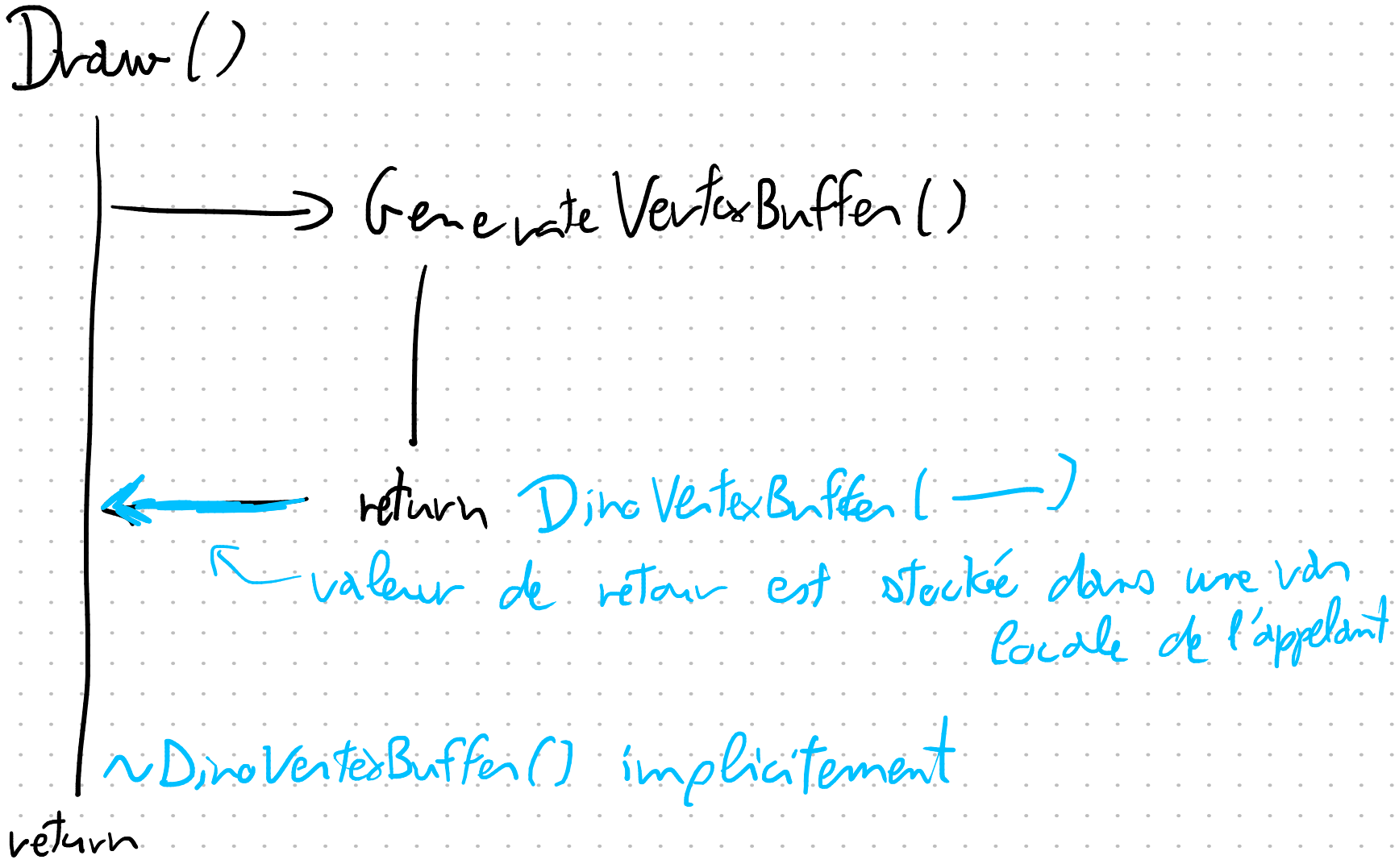
Le destructeur s'appelle "~" + le nom de la classe

Pas de type de retour

```
DinoPlayer :: ~DinoPlayer() {

}
```

9e) Dans DinoPlayer :



Problème avec

g_vbufID_prenom

→ Var globale → construite avant la fonction main() et GameInit()

→ Initialisée dans GameInit()

→ Détruire dans GameShut()

On peut pas se baser sur des variables locale

Solution 1. Rester avec uint64_T

2. std::optional<DinoVertexBuffer>

→ À l'init, on .emplace(——)

→ Au shut, on .reset()

Construction par copie

```
// Copie de vbuf
DinoVertexBuffer vbuf2 = vbuf;
// Par défaut, copier un type revient à copier tous ses membres
// vbuf2.m_vbufID = vbuf.m_vbufID
```

2 DinoVertexBuffer contiennent le même ID

```
// Destructeur de 'vbuf2' appelé implicitement par le compilateur
// Destructeur de 'vbuf' appelé implicitement par le compilateur
// Destructeur de 'vs' appelé implicitement par le compilateur
```

Il va être détruit deux fois

Assignation par copie

```
// à droite du = : on construit une variable temporaire
// puis on fait une copie
// puis on détruit la variable temporaire
vbuf = DinoVertexBuffer(...);
```

```
// Constructeur par copie
DinoVertexBuffer(DinoVertexBuffer const&) = delete;
```

Syntaxe pour constructeur par copie

Ces fonctions ne doivent pas être générées par le compilateur.

```
// Opérateur d'assignement par copie
DinoVertexBuffer& operator=(DinoVertexBuffer const& other) = delete;
```

Syntaxe pour opérateur d'assignement par copie

→ Faire une copie devient une erreur de compilation plutôt qu'un bug au runtime

Bonne pratique du C++ : Règle de 0 ou 3

Si on définit l'une de ces 3 méthodes :

- → Constructeur par copie
- → Assignation par copie
- → Destructeur

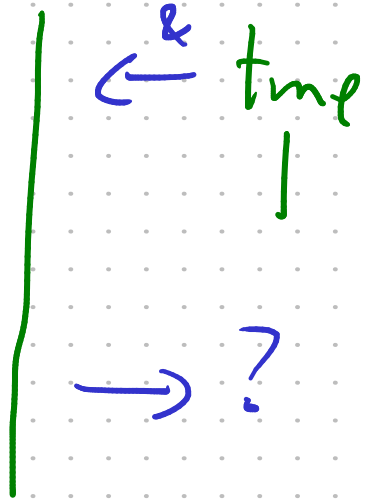
Alors il faut définir les 3 méthodes.
(ou en marquer delete)

Assignation par copie depuis une variable temporaire

```
// à droite du = : on construit une variable temporaire
// puis on fait une copie
// puis on détruit la variable temporaire
vbuf = DinoVertexBuffer(...);
```

Logiquement, ce n'est pas une copie, mais un transfert

vbuf



HORS PROGRAMME

Opérateurs de copie

→ Interdire une copie

"Copier" DinoVertexBuffer → - Créer une nouvelle allocation GPU
- Copier la mémoire d'un buffer à l'autre (GPU)
- Garder l'ancienne allocation GPU

Opérateurs de move (transfert de ressources)

→ Autoriser le transfert de ressources d'une instance à l'autre

"Move" DinoVertexBuffer → - Rien ne change sur le GPU
- Transférer la responsabilité de détruire le buffer

HORS PROGRAMME

```
// Constructeur par move
DinoVertexBuffer(DinoVertexBuffer&& other);

// Opérateur d'assignement par move
DinoVertexBuffer& operator=(DinoVertexBuffer&& other);
```

&& → référence vers variable temporaire

HORS PROGRAMME

Bonne pratique du C++ : Règle de 0 ou 3 ou 5

Si on définit l'une de ces méthodes :

Copie →
- → Destructeur
- → Constructeur par copie
- → Assignation par copie

} → Faut définir 3 ou 5 méthodes

Transfert de ressources →
- → Constructeur par move
- → Assignation par move

} → Faut définir les 5 méthodes